Version: 1.0



Selection

C-Libft

Résumé

Créer une bibliothèque statique en C (`libft.a`) et automatiser sa compilation à l'aide d'un Makefile.

#C #Library #Makefile



Avertissement relatif à la propriété intellectuelle

L'ensemble du contenu présenté dans ce module de formation — y compris, sans s'y limiter, les textes, images, graphiques et autres éléments — est protégé par les droits de propriété intellectuelle détenus par l'Association 42.

Conditions d'utilisation

- **Usage personnel :** L'utilisation du contenu de ce module est strictement réservée à un usage personnel. Toute reproduction, diffusion, modification, utilisation publique ou commerciale est interdite sans l'autorisation écrite préalable de l'Association 42.
- **Respect de l'intégrité :** Il est interdit de modifier, altérer ou adapter le contenu de manière à en dénaturer le sens ou à porter atteinte à son intégrité.

Protection des droits

Toute violation des présentes conditions constitue une atteinte aux droits de propriété intellectuelle et peut faire l'objet de poursuites. L'Association 42 se réserve le droit d'engager toute action nécessaire à la protection de ses droits, notamment par voie judiciaire et en réclamant des dommages et intérêts le cas échéant.

Pour toute question relative à l'utilisation du contenu ou pour obtenir une autorisation, veuillez contacter : `legal@42.fr`

Table des matières

1 Instructions 1

2 Préambule 5

3 Tutoriel 6

4 Exercice 0 : libft 9

5 Exercice 1 : Makefile 10

6 Rendu et évaluation par les pairs 12

Chapitre 1

Instructions

- Seule cette page servira de référence : ne faites pas confiance aux rumeurs.
- Attention ! Ce document pourrait potentiellement changer jusqu'à la soumission.
- Assurez-vous d'avoir les permissions appropriées sur vos fichiers et répertoires.
- Vous devez suivre les procédures de soumission pour tous vos exercices.
- Vos exercices seront vérifiés et notés par vos camarades de classe.
- De plus, vos exercices seront vérifiés et notés par un programme appelé Moulinette.
- Moulinette est très méticuleuse et stricte dans son évaluation de votre travail. Elle est entièrement automatisée, et il n'y a aucun moyen de négocier avec elle. Donc, pour éviter les mauvaises surprises, soyez aussi minutieux que possible.
- Moulinette n'est pas très ouverte d'esprit. Elle n'essayera pas de comprendre votre code s'il ne respecte pas la Norme. Moulinette s'appuie sur un programme appelé `norminette` pour vérifier si vos fichiers respectent la norme. En bref : il serait stupide de soumettre un travail qui ne passe pas la vérification de `norminette`.
- Ces exercices sont soigneusement disposés par ordre de difficulté - du plus facile au plus difficile. Nous `ne considérerons pas` un exercice plus difficile réussi si un exercice plus facile n'est pas parfaitement fonctionnel.
- Utiliser une fonction interdite est considéré comme de la tricherie. Les tricheurs obtiennent -42, et cette note est non négociable.
- Vous n'aurez à soumettre une fonction main() que si nous demandons un programme.
- Moulinette compile avec ces flags : -Wall -Wextra -Werror, et utilise `cc`.
- Si votre programme ne compile pas, vous obtiendrez `0`.
- Vous ne pouvez pas laisser aucun fichier supplémentaire dans votre répertoire autre que ceux spécifiés dans le sujet.
- Vous avez une question ? Demandez à votre pair à droite. Sinon, essayez votre pair à gauche.
- Votre guide de référence s'appelle `Google / man / Internet / ...`.
- Consultez le Slack Piscine.
- Examinez attentivement les exemples. Ils pourraient très bien appeler à des détails qui ne sont pas explicitement mentionnés dans le sujet...

© 2026 Association 42. - Tous droits réservés

- Par Odin, par Thor ! Utilisez votre cerveau ! ! !



N'oubliez pas d'ajouter l'*en-tête standard 42* dans chacun de vos fichiers .c/.h. Norminette vérifie son existence de toute façon !



Norminette doit être lancée avec le flag *-R CheckForbiddenSourceHeader*. Moulinette l'utilisera aussi.

© 2026 Association 42. - Tous droits réservés

Contexte

La Piscine C est intense. C'est votre premier grand défi à 42 — une plongée profonde dans la résolution de problèmes, l'autonomie et la communauté.

Durant cette phase, votre objectif principal est de construire vos fondations — à travers la lutte, la répétition, et surtout l'échange d'**apprentissage par les pairs**.

À l'ère de l'IA, les raccourcis sont faciles à trouver. Cependant, il est important de considérer si votre utilisation de l'IA vous aide vraiment à grandir — ou si elle se contente de vous empêcher de développer de vraies compétences.

La Piscine est aussi une expérience humaine — et pour l'instant, rien ne peut remplacer cela. Pas même l'IA.

Pour un aperçu plus complet de notre position sur l'IA — en tant qu'outil d'apprentissage, dans le cadre du programme ICT, et en tant qu'attente croissante sur le marché du travail — veuillez vous référer à la FAQ dédiée disponible sur l'intranet.

Message principal

- Construire des fondations solides sans raccourcis.
- Développer réellement des compétences techniques et personnelles.
- Expérimenter un véritable apprentissage par les pairs, commencer à apprendre à apprendre et à résoudre de nouveaux problèmes.
- Le parcours d'apprentissage est plus important que le résultat.
- Apprendre les risques associés à l'IA, et développer des pratiques de contrôle efficaces et des contre-mesures pour éviter les pièges courants.

Règles pour les apprenants :

- Vous devez appliquer le raisonnement à vos tâches assignées, surtout avant de vous tourner vers l'IA.
- Vous ne devez pas demander de réponses directes à l'IA.
- Vous devez apprendre l'approche globale de 42 sur l'IA.

© 2026 Association 42. - Tous droits réservés

Résultats de la phase :

Dans cette phase de fondation, vous obtiendrez les résultats suivants :

- Acquérir des bases techniques et de codage appropriées.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos activités. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne gaspillez pas votre temps (ou le nôtre) juste pour démontrer que l'IA peut résoudre le problème donné.
- Apprendre à 42 ne consiste pas à connaître la réponse — il s'agit de développer la capacité à en trouver une. L'IA vous donne la réponse directement, mais cela vous empêche de construire votre propre raisonnement. Et le raisonnement prend du temps, des efforts, et implique des échecs. Le chemin vers le succès n'est pas censé être facile.
- Gardez à l'esprit que lors des examens, l'IA n'est pas disponible — pas d'internet, pas de smartphones, etc. Vous réaliserez rapidement si vous avez trop compté sur l'IA dans votre processus d'apprentissage.
- L'apprentissage par les pairs vous expose à différentes idées et approches, améliorant vos compétences interpersonnelles et votre capacité à penser de manière divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne soyez pas timide — parlez, posez des questions, et apprenez ensemble !
- Oui, l'IA fera partie du programme — à la fois comme outil d'apprentissage et comme sujet en soi. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Afin d'en apprendre davantage sur notre approche crescendo, vous passerez par la documentation disponible sur l'intranet.

✓ Bonne pratique :

> Je suis bloqué sur un nouveau concept. Je demande à quelqu'un à proximité comment il l'a abordé. Nous parlons pendant 10 minutes — et soudain, ça fait clic. Je comprends.

✗ Mauvaise pratique :

> J'utilise secrètement l'IA, je copie du code qui semble correct. Pendant l'évaluation par les pairs, je ne peux rien expliquer. J'échoue. Pendant l'examen — sans IA — je suis à nouveau bloqué. J'échoue.

© 2026 Association 42. - Tous droits réservés

Chapitre 2

Préambule

Recette du poulet à l'ail 42 gousses

- **Ingrédients :**
  - 1 poulet entier découpé en 8 morceaux
  - 42 gousses d'ail (oui, 42 !)
  - 1/2 tasse d'huile d'olive
  - Le jus d'un citron
  - 1 c. à soupe de thym séché
  - 1 c. à soupe de romarin séché
  - Sel et poivre selon votre goût
- **Instructions :**

1. Préchauffez le four à 200°C (400°F).
2. Dans un grand bol, mélangez l'huile d'olive, le jus de citron, le thym, le romarin, le sel et le poivre.
3. Ajoutez les morceaux de poulet et mélangez pour bien les enrober.
4. Placez le poulet dans un plat à gratin et glissez les 42 gousses d'ail autour des morceaux.
5. Versez le reste de la marinade sur le poulet.
6. Faites cuire au four pendant 45 à 50 minutes, jusqu'à ce que le poulet soit doré et bien cuit.
7. Servez chaud, avec du pain croustillant pour tartiner l'ail confit.

© 2026 Association 42. - Tous droits réservés

Chapitre 3

Tutoriel

- Maintenant que vous allez commencer à utiliser votre propre bibliothèque que vous venez de créer, incluant de plus en plus de fichiers `.c`, compiler tout depuis le début pour obtenir votre logiciel entièrement opérationnel prendra de plus en plus de temps et de commandes à exécuter. En tant que programmeurs, utilisons un programme pour faire le travail fastidieux à notre place.
- Le programme que nous allons utiliser est **make**. Il vient avec un ensemble de fonctionnalités qui nous permettront de faire facilement toutes les tâches nécessaires, depuis le code source jusqu'au produit final que nous visons. Le programme **make**, qui fait partie du système, s'attend à trouver des instructions dans un fichier spécifique appelé **Makefile**. Il suffit de taper `make` dans le terminal pour effectuer toutes les actions nécessaires à la création de votre logiciel complet.
- Aujourd'hui, vous allez créer votre tout premier **Makefile**, comprendre sa syntaxe, et découvrir un très petit sous-ensemble de fonctionnalités pour atteindre nos objectifs actuels. À vous dans le futur de révéler et d'utiliser des fonctionnalités plus complexes ou avancées de **make** et **Makefile**.

  L'utilisation d'un **Makefile** implique que vous utilisez une fonctionnalité de compilation appelée "compilation séparée". Vous trouverez de nombreuses explications à ce sujet. Basiquement, au lieu de créer directement votre logiciel à partir des fichiers `.c`, vous ferez une pause à une étape intermédiaire : les fichiers objets (`.o`). Il s'agit déjà de code compilé, mais seulement pour les fonctions présentes dans le fichier `.c`. Vous ne pouvez pas l'utiliser tel quel, sauf en l'assemblant avec d'autres fichiers objets pour créer votre logiciel complet. Tout comme les pièces d'un puzzle. Le principal avantage de la compilation séparée est d'éviter de recompiler un fichier `.c` qui n'a pas été modifié depuis la dernière compilation. Pour les projets ENORMES (comme la compilation de Google Chromium qui peut prendre beaucoup de temps), c'est définitivement nécessaire.
- Créez un répertoire de travail pour ce tutoriel et placez-y votre fichier `ft_strdup.c` et un fichier `main.c` qui teste cette fonction. Si vous n'avez pas ces fichiers, créez-les maintenant. Assurez-vous qu'ils compilent et fonctionnent correctement avec une compilation manuelle avant de continuer.
- Dans ce même répertoire, créez un fichier nommé exactement `Makefile` (attention à la casse, pas d'extension). Ouvrez-le avec votre éditeur de texte.
- Maintenant, écrivons notre premier **Makefile**. Un **Makefile** simple et classique est structuré en deux parties : les déclarations et les règles. Les déclarations sont l'endroit où vous définissez plusieurs éléments, tout comme les variables d'environnement du shell.

© 2026 Association 42. - Tous droits réservés



La syntaxe est : **NOM_VAR = chaîne**. Habituellement, le nom de la variable à gauche est en majuscules. Si vous n'êtes pas sûr, vérifiez avec vos voisins.

Commencez par taper ces lignes dans votre Makefile :

Makefile

```
NAME = test_strdup
SOURCES = ft_strdup.c main.c
OBJECTS = $(SOURCES:.c=.o)
```

Cette partie correspond aux déclarations.

- Ajoutons une règle. Ajoutez ce qui suit dans votre **Makefile** :

Makefile

```
$(NAME): $(OBJECTS)
    cc -o $(NAME) $(OBJECTS)
```

Par défaut, lorsque vous tapez **make**, seule la toute première règle sera exécutée.

- **make** vient avec des règles implicites. La plus utilisée compile automatiquement tout fichier **.c** en un fichier **.o**. Enregistrez votre `Makefile`, puis tapez `make` dans votre terminal et observez ce qui se passe.
- Tapez `ls` pour voir les fichiers créés. Vous devriez voir des fichiers **.o** et votre exécutable. Exécutez votre exécutable pour tester s'il fonctionne.
- Ajoutez une règle **clean** qui supprime tous les fichiers .o.
- Tapez `make clean` puis `ls` pour vérifier que les fichiers **.o** ont été supprimés.
- Ajoutez une règle **fclean** qui appelle la règle **clean**, puis supprime le programme.
- Tapez `make clean` puis listez votre répertoire. Tout devrait être nettoyé. Recompilez avec `make` pour vérifier que tout fonctionne encore.
- Ajoutez des **flags de compilation** en ajoutant cette ligne après vos variables :

Makefile

```
CFLAGS += -Wall
```

Nettoyez tout avec `make fclean` puis recompilez avec `make`. Observez comment l'option -Wall apparaît automatiquement dans la compilation.

- Créez un sous-répertoire appelé `ft_range` et placez-y votre fichier `ft_range.c` et un `main.c`. Modifiez maintenant votre `Makefile` pour gérer les deux fonctions.

© 2026 Association 42. - Tous droits réservés

Tapez simplement `make` pour compiler à la fois `ft_strdup` et `ft_range`. Vous pouvez utiliser des sous-répertoires. Testez les deux exécutables pour vérifier qu'ils fonctionnent.

- Expérimentez avec le comportement de **make**. Supprimez un fichier **.o** avec `rm ft_strdup.o`, puis tapez `make all`. Observez quels fichiers sont recompilés et lesquels ne le sont pas.
- Tapez `make all` une deuxième fois sans rien changer. Que se passe-t-il et pourquoi ?



Vous n'êtes pas sûr de la façon dont **make** décide quoi recompiler ? Recherchez les **dépendances de make** et les **horodatages de fichiers** pour comprendre ce mécanisme, puis vérifiez votre compréhension avec votre voisin.

- Ajoutez l'option **-Wall** à la compilation de chaque fichier **.c**.
- Assurez-vous que les règles **clean** et **fclean** s'appliquent à la fois à `ft_strdup` et `ft_range`.

© 2026 Association 42. - Tous droits réservés

Chapitre 4

Exercice 0 : libft

	Exercice: 0	
libft		
Répertoire: *ex*0/		
Fichiers à Rendre: `libft_creator.sh, ft_putchar.c, ft_swap.c, ft_putstr.c, ft_strlen.c, ft_strcmp.c`		
Autorisé: `write`		

- Créez votre propre bibliothèque `ft`. Elle s'appellera `libft.a`.
- Un script shell appelé `libft_creator.sh` devra compiler les fichiers source de manière appropriée et créer votre bibliothèque.
- Cette bibliothèque devra contenir toutes les fonctions suivantes :

Déclaration

```
void    ft_putchar(char c);
void    ft_swap(int *a, int *b);
void    ft_putstr(char *str);
int     ft_strlen(char *str);
int     ft_strcmp(char *s1, char *s2);
```

- Nous exécuterons la commande suivante :

Terminal :

```
?> sh libft_creator.sh
```

© 2026 Association 42. - Tous droits réservés

Chapitre 5

Exercice 1 : Makefile

	Exercice: 1	
Makefile		
Répertoire: *ex*1/		
Fichiers à Rendre: `Makefile`		
Autorisé: `None`		

- Créez un `Makefile` qui compilera une bibliothèque `libft.a`.
- Votre Makefile devra afficher toutes les commandes exécutées.
- Il ne devra exécuter aucune commande inutile.
- Le `Makefile` ira chercher ses fichiers sources dans le dossier "srcs".
- Ces fichiers seront : ft_putchar.c, ft_swap.c, ft_putstr.c, ft_strlen.c, ft_strcmp.c
- Le `Makefile` ira chercher ses fichiers headers dans le dossier "includes".
- Ces fichiers seront : ft.h
- Il devra compiler les fichiers .c avec `cc` et les options `-Wall -Wextra -Werror`, dans cet ordre.
- La bibliothèque devra se trouver à la racine de l'exercice.
- Les fichiers .o devront se trouver à côté de leur fichier .c correspondant.
- Le `Makefile` devra également inclure les règles suivantes : `clean`, `fclean`, `re`, `all`, et bien sûr `libft.a`.
- Un fichier `main.c` à la racine devra également être compilé avec les mêmes flags. Son fichier .o devra être inclus dans la bibliothèque.
- L'exécution de `make` seule devra être équivalente à `make all`.
- La règle `all` devra être équivalente à `make libft.a`.
- La règle `clean` devra supprimer tous les fichiers temporaires générés.
- La règle `fclean` devra faire la même chose que `make clean` mais aussi supprimer tous les fichiers binaires créés par `make all`.

© 2026 Association 42. - Tous droits réservés

- La règle `re` devra faire un `make fclean` suivi d'un `make all`.
- Votre Makefile ne devra jamais recompiler un fichier sans raison.
- Nous ne récupérerons que votre Makefile et nous le testerons avec nos propres fichiers.



Attention aux jokers (`wildcards`) !

© 2026 Association 42. - Tous droits réservés

Chapitre 6

Rendu et évaluation par les pairs

Déposez votre devoir dans votre dépôt `Git` comme d'habitude. Seul le contenu de ce dépôt sera évalué pendant la soutenance. N'hésitez pas à revérifier les noms de vos fichiers pour vous assurer qu'ils sont corrects.



Vous ne devez rendre que les fichiers demandés par le sujet de ce projet.

© 2026 Association 42. - Tous droits réservés